

Library Management System

Final Project — C# Console Application + SQL Server + Entity Framework Core

1. Project Overview

You are required to build a **Library Management System** as a Console Application.

The system will allow the user to manage books, categories, members, and borrowing operations.

The project must use:

- C#
- Object-Oriented Programming
- Collections
- LINQ
- SQL Server
- Entity Framework Core
- Migrations
- Relationships between entities

The application should store its data in a SQL Server Database instead of keeping the data only in memory.

2. Project Entities

Book

The Book entity represents a book in the library. It should contain at least:

- Id
- Title
- Author
- Price
- PublishedYear
- CategoryId

A Book belongs to one Category.

Category

The Category entity represents a category of books. It should contain at least:

- Id
- Name

One Category can contain multiple Books.

Category 1 ----- * Book

Member

The Member entity represents a library member. It should contain at least:

- Id
- Name
- Email
- Phone

A Member can have multiple borrowing operations.

Borrowing

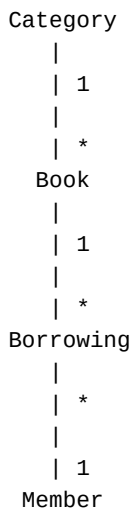
The Borrowing entity represents a book borrowing operation. It should contain at least:

- Id
- BookId
- MemberId
- BorrowDate
- ReturnDate

A Borrowing belongs to one Book and one Member.

Book 1 ----- * Borrowing
Member 1 ----- * Borrowing

3. Database Relationships



Make sure that the relationships are correctly represented in your C# classes and Entity Framework Core model.

4. Entity Framework Core Setup

Create an ApplicationDbContext class that inherits from DbContext.

The Context should contain a DbSet for each entity:

- Books
- Categories
- Members
- Borrowings

The database connection should be stored in **appsettings.json**. Use the ConnectionStrings section to store the SQL Server connection string.

The EF Core configuration should be implemented inside ApplicationDbContext.

5. Database Creation

After creating the entities and configuring the ApplicationDbContext:

- Create the initial migration.
- Apply the migration to the database.
- Verify that the database and tables were created successfully.

Use EF Core migrations to manage database changes.

6. Main Menu

When the application starts, display a menu similar to:

```
===== Library Management System =====
```

1. Book Management
2. Category Management
3. Member Management
4. Borrowing Management
5. Reports
6. Exit

The user should be able to choose an operation from the menu.

7. Book Management

Add Book

Allow the user to enter Title, Author, Price, Published Year, and Category. The book should then be stored in the database.

Update Book

Ask the user for the Book Id. If the book exists, allow the user to modify its information. If the book does not exist, display an appropriate message.

Delete Book

Ask the user for the Book Id. If the book exists, delete it from the database. Handle the case where the book does not exist.

Get Book By Id

Ask the user for a Book Id and display the book information. The result should include the Category information.

Get All Books

Display all books with their Id, Title, Author, Price, Published Year, and Category.

8. Search and Filtering

Implement LINQ queries to allow the user to search and filter books.

- **Search by Title:** Allow the user to enter part of a book title and display matching books.
- **Filter by Price:** Display books cheaper than or more expensive than a specific price.
- **Filter by Published Year:** Display books published after a specific year.
- **Sort Books:** Sort by Title, Price ascending, and Price descending.

Use LINQ to implement these operations.

9. Category Management

Implement:

- Add Category
- Update Category
- Delete Category
- Get Category By Id
- Get All Categories

When displaying a Category, you should be able to display the books that belong to it.

10. Member Management

Implement:

- Add Member
- Update Member
- Delete Member
- Get Member By Id
- Get All Members

When displaying a Member, display the member's basic information. You should also be able to display the member's borrowing history.

11. Borrowing Management

Borrow Book

The user should enter Member Id and Book Id.

Before creating the borrowing operation, the system must check that:

- The Member exists.
- The Book exists.
- The Book is currently available.

If all conditions are satisfied, create a new Borrowing record, store the current date as BorrowDate, and leave ReturnDate empty initially.

Return Book

Allow the user to return a borrowed book. When the book is returned, set the ReturnDate. The book should become available for borrowing again.

The system should not allow a book that has already been returned to be returned again.

12. Borrowing History

Member Borrowing History

Given a Member Id, display Member Name, Book Title, Borrow Date, and Return Date.

Book Borrowing History

Given a Book Id, display all borrowing operations related to that book.

Use Entity Framework Core and LINQ to retrieve the related data.

13. Reports

Most Borrowed Books

Display books ordered by the number of times they have been borrowed.

Book	Borrow Count

Clean Code	12
C# in Depth	9
Database Fundamentals	6

Use GroupBy, Count, and OrderByDescending.

Number of Books Per Category

Display the number of books in each category.

Category	Number of Books

Programming	15
History	8
Science	10
Novel	20

Average Book Price

Display the average price of all books. Use the appropriate LINQ aggregate function.

Most Active Members

Display members ordered by the number of books they have borrowed.

Member	Borrowing Count

Ahmed Ali	10
Mona Ahmed	8
Omar Hassan	5

14. LINQ Requirements

The project must demonstrate the use of LINQ. Use appropriate LINQ operators such as:

- Where
- Select
- OrderBy
- OrderByDescending
- FirstOrDefault
- Any
- Count
- Average
- GroupBy

Use the operator that best fits each requirement. Do not write all queries in the same way.

15. Entity Framework Core Requirements

DbContext

Create and use an ApplicationDbContext.

DbSet

Create a DbSet for each entity.

Configuration

Read the SQL Server connection string from appsettings.json.

Relationships

Implement the required relationships between entities.

Migrations

Use Add-Migration and Update-Database to create and update the database.

CRUD

Implement Create, Read, Update, and Delete for the main entities.

Include

Use Include when you need to retrieve related data, such as Book → Category or Member → Borrowing → Book.

16. Console Application Requirements

The application must be a Console Application. The user should interact with the system through the console.

- Display clear menus.
- Read user input.
- Validate important inputs.
- Display clear success/error messages.
- Return the user to the appropriate menu after each operation.

Avoid putting the entire application inside Main. Create separate methods for different operations.

```
AddBook()  
UpdateBook()  
DeleteBook()  
GetBookById()  
GetAllBooks()  
SearchBooks()  
BorrowBook()  
ReturnBook()
```

You are free to organize the methods in a way that makes the code clean and readable.

17. Data Validation

The application should handle invalid user input. At minimum, consider:

- Invalid Id
- Non-existing Book
- Non-existing Member
- Non-existing Category
- Invalid price
- Invalid year
- Borrowing an unavailable book
- Returning an already returned book

The application should not crash because of normal invalid user input.

18. Important Rules

- All persistent data must be stored in SQL Server.
- Do not use a hard-coded list as the main data source.
- Use Entity Framework Core for database operations.
- Use LINQ for querying data.
- Use relationships between entities.
- Use migrations to create/update the database.

- Use SaveChanges() when appropriate to persist changes.
- Do not write raw SQL queries for the required operations.
- Keep the code organized into methods.
- Use meaningful names for classes, properties, and methods.

19. Challenge

After completing all the previous requirements, add at least **two additional features** of your own.

Examples:

- Show currently borrowed books.
- Show currently available books.
- Find the most popular category.
- Search members by name or email.
- Show books that have never been borrowed.
- Show members who currently have borrowed books.
- Add a maximum borrowing limit for each member.
- Add a late-return calculation.
- Add a fine for late returns.

The additional features must use the concepts learned during the course.

20. Final Goal

By the end of the project, your application should allow a user to manage the library's books, members, categories, and borrowing operations.

```

C#
↓
OOP
↓
Collections
↓
LINQ
↓
SQL
↓
Relationships
↓
Entity Framework Core
↓
SQL Server

```

21. Deliverables

- Complete Visual Studio Project.
- SQL Server Database created through EF Core Migrations.
- All Entity Classes.

- ApplicationDbContext.
- appsettings.json.
- Migration files.
- Complete Console Application.
- A short README explaining the project idea, entities, relationships, main features, and how to run the project.